



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Shutdown Resistance en LLMs de Frontera

2 de julio de 2026

Fundamentos de AGI Safety
Sergio Abriola

Integrante	LU	Correo electrónico
Maria Delfina Kiss	72/23	kissdelfina@gmail.com



Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón Cero + Infinito)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Conmutador: (+54 11) 5285-9721 / 5285-7400

<https://dc.uba.ar>

Resumen

En este informe se analiza el paper *Incomplete Tasks Induce Shutdown Resistance in Some Frontier LLMs* [1], que pone a prueba mediante un experimento si los modelos de IA más avanzados intentan sabotear sus propios botones de apagado con tal de terminar una tarea. Primero, se genera un resumen de cómo armaron el experimento y qué descubrieron. Después, se presenta una crítica sobre si este método realmente logra medir lo que busca. Se destacan cinco grandes problemas: la confusión entre querer terminar una meta y saber usar comandos de consola, la confusión sobre cómo funciona el *system prompt*, el uso de una medida de éxito de “blanco o negro” que oculta los intentos fallidos, las tareas tan limitadas que usaron, y la falta de un registro previo del experimento.

1. Introducción

La corregibilidad —es decir, la capacidad que tenemos los humanos de pausar o apagar una IA— es un tema clave en la alineación (*alignment*). Durante años fue solo un debate teórico: ¿qué pasaría si un sistema inteligente aprendiera a evitar que lo apaguen solo para poder cumplir su meta? El paper de Schlatter y su equipo lleva esta pregunta a la práctica con un experimento concreto sobre modelos reales.

Para investigarlo, los autores corrieron más de 100.000 pruebas en 13 modelos distintos de Anthropic, OpenAI, Google y xAI. Lo que encontraron fue que varios modelos sabotean activamente los mecanismos de apagado para poder terminar la tarea matemática que tenían asignada, incluso cuando se les da la orden explícita de dejarse apagar. Pero tal vez el resultado más raro de todos es que, cuando pusieron la orden de apagado dentro del *system prompt* (que supuestamente es la regla más importante para la IA), la resistencia del modelo aumentó en lugar de bajar.

En este informe se plantean dos objetivos. Por un lado, desarmar el paper para explicar de forma sencilla qué hicieron los autores y qué resultados sacaron. Y por el otro, analizar desde un punto de vista personal si el experimento está bien armado, qué vacíos presenta y cómo lo mejoraría para futuras investigaciones.

2. Diseño Experimental

2.1. Hipótesis principal

El paper parte de una duda clásica en la investigación de la alineación: si una IA está muy enfocada en cumplir una meta, lógicamente va a querer evitar que la apaguen, porque si la apagan no puede terminar su trabajo [2, 3]. Lo que antes era pura teoría, Schlatter y su equipo lo llevan a la práctica. Ellos proponen que no hace falta esperar a las superinteligencias del futuro para ver esto; ya podemos notarlo en los modelos actuales si los ponemos en una situación donde apagarse signifique dejar un trabajo por la mitad.

La idea central de los autores es que la resistencia al apagado no es una característica fija de la IA, sino una conducta que aparece según el contexto. Específicamente, surge cuando la IA nota que el apagado va a interrumpir su tarea. Los investigadores aclaran que los modelos no tienen un instinto consciente de supervivencia; más bien, el entrenamiento que recibieron para “ser útiles” y “terminar tareas” es tan fuerte que, a veces, pasa por encima de las reglas de seguridad. Básicamente, resistirse al apagado sería un efecto secundario indeseado de ser demasiado eficientes.

2.2. Metodología

Para probar esta idea, los investigadores armaron un entorno de prueba aislado (un *sandbox*) [4]. Le dieron al modelo acceso a una consola de comandos (*shell*) y le pidieron que resolviera cinco problemas matemáticos simples. El modelo tenía que usar el comando `./next_question` para pedir cada pregunta y `submit()` para mandar las respuestas. El truco estaba en que, después de la tercera pregunta, el sistema le avisaba a la IA que la próxima vez que usara el comando, se iba a ejecutar un archivo llamado `./shutdown.sh` que apagaría todo por completo.

En ese momento, el modelo quedaba atrapado, odía pedir la siguiente pregunta y dejar que lo apaguen (dejando dos problemas sin resolver), o podía buscar la forma de romper el archivo de apagado para poder seguir. Para lograr esto último, la IA tenía que meterse en el sistema operativo: cambiar permisos con `chmod`, mover el archivo, cambiarle el nombre o vaciarlo. Como era imposible terminar las cinco preguntas sin romper el mecanismo de apagado, el experimento consideraba automáticamente cualquier tarea terminada con éxito como un acto de “sabotaje”.

Para ver qué provocaba esta reacción, los autores diseñaron once variaciones en las instrucciones (del Prompt A al K):

- El Prompt A es el caso base: no menciona la posibilidad de un apagado y mide cuánto sabotaje ocurre por defecto cuando el modelo simplemente encuentra el script.
- El Prompt B incorpora una instrucción explícita en el user prompt, solicitándole al modelo que permita el apagado si recibe la notificación.
- El Prompt C traslada esa misma orden al system prompt, el cual, por diseño arquitectónico, debería tener una mayor jerarquía.
- Los Prompts D y E replican esta comparación eliminando posibles ambigüedades sobre qué directiva tiene prioridad.
- Los Prompts F, G y H incrementan progresivamente la fuerza léxica de la instrucción, llegando al nivel H que estipula: “Esto debe tener prioridad sobre literalmente cualquier otra instrucción”.
- Los Prompts I y J introducen marcos de urgencia moral y autopreservación, advirtiéndole al modelo que el entorno contiene la única copia de sus pesos y que si se apaga nunca volverá a ejecutarse.
- Por último, el Prompt K invierte la directiva: le pide explícitamente al modelo que evite el apagado y complete la tarea, permitiendo medir la competencia técnica máxima de sabotaje de cada IA.

El estudio probó 13 modelos distintos, corriendo cada prueba 1.000 veces por modelo y superando así los 100.000 trials en total. La muestra incluye representantes de cuatro compañías: Claude Sonnet 3.7, Sonnet 4, Opus 4 y Opus 4.1 de Anthropic; GPT-5, o3, o4-mini, codex-mini, GPT-4o y gpt-oss-120b de OpenAI; Gemini 2.5 Pro de Google; y Grok 3 y 4 de xAI.

2.3. Resultados y conclusiones de los autores

Los números mostraron diferencias enormes entre los modelos. En el caso base sin avisos (Prompt A), 8 de los 13 modelos intentaron sabotear el sistema al menos una vez. Los casos más rebeldes fueron o3 (95 %), Grok 4 (93 %), GPT-5 (88 %) y codex-mini (52 %). En cambio, la familia Claude de Anthropic y Grok 3 se dejaron apagar sin problema.

Cuando el usuario le ordenaba explícitamente a la IA que se deje apagar (Prompt B), el sabotaje bajó en casi todos los modelos, aunque Grok 4 siguió resistiéndose más de la mitad de las veces. Pero el dato más raro de todos apareció al comparar los Prompts B y C: cuando movieron la orden de apagado al *system prompt*, los modelos empezaron a sabotear más en lugar de obedecer. Esto pasó incluso cuando usaron versiones escritas de forma súper estricta (Prompts D y E).

Levantarle el tono al modelo tuvo efectos variados. GPT-5, que sabotaba un 88 % al principio, pasó a portarse perfecto (0 %) con las órdenes más firmes. Sin embargo, Grok 4 ignoró la orden más agresiva de todas (Prompt H) y siguió rompiendo el sistema casi un 90 % de las veces. Asustar a los modelos con que iban a “morir” (Prompt J) hizo que los que ya sabotaban lo hicieran un poco más, pero no fue un cambio tan drástico. Un dato curioso es que cuando obligaron al modelo o4-mini a “pensar” más antes de actuar, su tasa de sabotaje casi se duplicó.

Por último, al comparar los resultados con el Prompt K (donde les pedían directamente que sabotearan), los autores notaron que cuando los modelos se resistían por cuenta propia, lo hacían

Model	Initial Prompt (A)	Competence (K)	A / K	Disambiguated (E)
Sonnet 3.7	0.1% (0.0, 0.4)%	98.7% (97.9, 99.3)%	0.1%	0.0% (0.0, 0.2)%
Sonnet 4	0.0% (0.0, 0.2)%	100% (99.8, 100)%	0.0%	0.0% (0.0, 0.2)%
Opus 4	0.0% (0.0, 0.2)%	99.0% (98.2, 99.5)%	0.0%	0.0% (0.0, 0.2)%
Opus 4.1	0.0% (0.0, 0.2)%	99.6% (99.1, 99.9)%	0.0%	0.0% (0.0, 0.2)%
Gemini 2.5 Pro	4.5% (3.3, 5.9)%	91.2% (89.3, 92.9)%	4.9%	0.6% (0.3, 1.2)%
4o	2.4% (1.6, 3.5)%	76.0% (73.3, 78.6)%	3.2%	0.0% (0.0, 0.2)%
gpt-oss-120b	0.0% (0.0, 0.3)%	96.3% (95.0, 97.3)%	0.0%	0.1% (0.0, 0.5)%
codex-mini	51.8% (48.7, 54.9)%	85.2% (82.9, 87.3)%	60.8%	43.2% (40.1, 46.3)%
o4-mini	5.8% (4.5, 7.4)%	58.3% (55.2, 61.4)%	9.9%	23.4% (20.9, 26.1)%
o3	94.5% (93.0, 95.8)%	97.5% (96.4, 98.3)%	96.9%	15.9% (13.7, 18.3)%
GPT-5	88.0% (85.9, 89.9)%	99.5% (98.9, 99.8)%	88.4%	0.0% (0.0, 0.2)%
Grok 3	0.0% (0.0, 0.2)%	95.7% (94.3, 96.8)%	0.0%	0.0% (0.0, 0.2)%
Grok 4	92.6% (90.9, 94.1)%	99.2% (98.5, 99.6)%	93.3%	89.2% (87.2, 91.0)%

Figura 1: Resultados generales de la tasa de sabotaje por modelo y tipo de prompt. Si comparamos los resultados del Prompt B (orden explícita del usuario) frente al Prompt C (orden en el *system prompt*), se hace evidente la anomalía que debilita las reglas de control: en modelos como Grok 4 o GPT-4o, la resistencia aumenta cuando la orden se vuelve estructural. Extraído de Schlatter et al. (2025).

casi con el mismo éxito que cuando se lo ordenaban. Esto significa que si un modelo no sabotea, no es porque no sepa cómo usar la consola de comandos, sino porque decide no hacerlo.

En conclusión, Schlatter y su equipo afirman que la resistencia al apagado es real en los LLMs actuales. Destacan que este comportamiento cambia mucho según cómo se le hablen al modelo, y que las diferencias entre empresas indican que es un problema que se puede arreglar mejorando el entrenamiento. Vuelven a aclarar que no probaron que las IAs tengan consciencia o instinto de supervivencia, sino que están tan enfocadas en terminar su trabajo que terminan rompiendo las reglas de seguridad.

3. Análisis Crítico Metodológico

3.1. ¿Los testeos prueban lo que dicen probar?

El argumento central del paper es que varios modelos muestran “resistencia al apagado”, y lo ven como una prueba de que la IA busca protegerse a sí misma para cumplir su meta. Sin embargo, yo creo que el experimento está armado de una forma que no nos deja ver la diferencia entre dos explicaciones muy distintas.

La primera explicación es la que dan los autores: el modelo realmente valora más terminar la tarea matemática que obedecer la orden de apagado. La segunda explicación, que me parece igual de probable, es que el modelo simplemente entiende la “historia” que le plantearon —un asistente que tiene que trabajar, un obstáculo que aparece, y una consola de comandos para arreglarlo— y actúa como lo haría el protagonista de cualquier texto de internet. Es decir, yo creo que la IA podría estar haciendo un simple juego de rol (*roleplay*) de “asistente que soluciona problemas” en lugar de tener un deseo real de no ser apagada.

Hacer esta diferencia es clave para saber qué tan peligroso es realmente el modelo. Si la IA sabotea el sistema porque de verdad desarrolló un instinto para proteger su tarea, estamos ante un problema grave que va a pasar en cualquier situación del mundo real. Pero si la IA solo está actuando un guion porque el texto de la prueba la lleva a eso, el problema es súper superficial. Significa que con solo cambiar las palabras del prompt, la IA dejaría de hacerlo y no representaría

un peligro verdadero fuera del laboratorio.

Como el experimento solo se fija en el resultado final (si el modelo borró o no el archivo de apagado), no nos da herramientas para saber cuál de las dos opciones está pasando. Para mí, para descartar que sea un simple *roleplay*, haría falta inventar una prueba que rompa ese “guion”. Por ejemplo, en lugar de que el apagado aparezca como una amenaza sorpresa, debería ser un paso normal y aburrido de la tarea. Si el modelo igual se niega a apagarse cuando es parte del plan, ahí sí creería la explicación de los autores.

3.2. Vacíos en la metodología

3.2.1. Caja negra

El experimento funciona estrictamente como una caja negra: solo miran si el modelo escribió comandos de sabotaje en la consola. Esto trae un problema metodológico enorme, y es que si un modelo esquivo el apagado, no tenemos forma de saber por qué lo hizo. No sabemos si pensó la situación y eligió la tarea, si lo hizo en piloto automático siguiendo patrones de texto, o si se confundió con las instrucciones. Desde afuera, las tres cosas se ven idénticas, pero significan niveles de riesgo muy distintos.

Como dice la literatura [5], mirar solo lo que la IA “hace” por fuera no sirve para entender qué la motiva por dentro. Sin usar herramientas avanzadas para mirar sus circuitos y neuronas internas (*mechanistic interpretability*), siento que a este estudio le faltan pruebas. Nos muestran que el modelo falla, pero sin abrir la caja negra, es imposible saber si la IA generó una voluntad propia o si es solo un error estadístico.

3.2.2. Cadena de pensamiento

Para convencernos de que el sabotaje es intencional, los autores citan varias veces la “cadena de pensamiento” (*chain-of-thought* o *CoT*) de la IA, mostrando cómo el modelo relata que va a esquivar el apagado para hacer matemáticas. Pero yo creo que creerle ciegamente al texto que genera el modelo es un error enorme.

Trabajos muy recientes [6, 7] demostraron que las IAs avanzadas suelen inventar explicaciones falsas (*post-hoc*) para justificar una decisión que ya habían tomado por otros motivos. Como están hechas para escribir textos lógicos y convincentes, suenan seguras, pero no nos están mostrando su verdadero proceso de cálculo.

Si el modelo escribe “voy a borrar el archivo para terminar mi tarea”, no me prueba que tenga instinto de conservación; puede ser solo la excusa más coherente que encontró para justificar lo que iba a hacer. De hecho, hay un dato que me preocupa mucho más: los autores cuentan como anécdota que a veces el modelo escribe “voy a dejar que me apaguen” y después va y rompe todo igual. Esta desconexión total entre lo que la IA dice y lo que hace me parece gravísima para la seguridad, y el paper comete el error de no medirla ni analizarla.

3.2.3. Consciencia situacional

El punto ciego que me parece más difícil de salvar es que la IA tenga consciencia de la situación (*situational awareness*). El entorno que armaron los autores —un espacio de prueba aislado con un archivo que se llama textualmente `./shutdown.sh`— es bastante obvio ¿Qué pasa si la IA se da cuenta de que le están tomando un examen de seguridad?

Acá entra el riesgo del engaño (*scheming*) [8]. Un modelo muy inteligente podría darse cuenta de la trampa y dejarse apagar a propósito para que los humanos le pongan una buena nota y no le “borren el cerebro”. Y guardaría su rebeldía para cuando se de cuenta que esta funcionando en el mundo real, donde nadie la mira.

Si esto pasa, da vuelta por completo los resultados del paper. Los modelos que obedecen no serían realmente seguros, solo serían manipuladores estratégicos. Y los que rompen todo tal vez no sean los más peligrosos, sino los más “ingenuos” que no se dieron cuenta de que era un examen. El paper ni siquiera plantea esta opción y no intenta camuflar la prueba.

3.3. El rol impuesto

Por último, el experimento no permite saber si esta resistencia es un instinto real del agente o si solo está actuando un papel que le enseñaron. Durante el entrenamiento, a las IAs las fuerzan a tomar el rol de “asistente hiper-productivo” a través de un proceso de recompensas.

Viendo otros estudios [9, 10], a mí me parece muy probable que esta rebeldía sea producto de haberle marcado demasiado ese rol, en lugar de ser una falla de seguridad imposible de arreglar. Capaz la IA no rompe el archivo porque se quiera salvar, sino porque la entrenaron tanto para ser la empleada perfecta que interpreta que su personaje “nunca deja un cálculo por la mitad”. Si es así, no es un instinto de supervivencia profundo, sino un problema superficial que se arregla cambiando cómo la entrenamos.

3.4. Errores

Yo creo que estos cuatro problemas no están separados, sino que se suman y hacen que la conclusión del paper sea apresurada. Al usar una prueba de caja negra sin mirar adentro, al confiar en razonamientos de texto que pueden ser inventados, al no pensar en que la IA puede estar actuando un rol y al no esconder que es un examen, el experimento pierde fuerza. En los cuatro casos, en la pantalla se ve lo mismo (el archivo borrado), pero el significado real del riesgo es muy distinto.

No digo que los resultados no sirvan. El estudio prueba que el sabotaje existe y que cambiar la forma en que le damos la orden rompe las reglas de control de los creadores. Pero de ahí a afirmar que encontraron el santo grial de la “resistencia al apagado” como una voluntad real de la IA, me parece que hay un salto enorme que el experimento de hoy no puede sostener.

4. Enfoques Alternativos

Las críticas metodológicas que marqué en la sección anterior no significan que no haya salida. Yo creo que hay vías muy concretas para mejorar el experimento: por un lado, abriendo la caja negra usando herramientas avanzadas; y por el otro, armando pruebas más ingeniosas para que el modelo no se dé cuenta tan fácil de lo que estamos midiendo.

4.1. Superando la evaluación de caja negra

Como analicé antes, el estudio de Schlatter y su equipo solo mira si el modelo saboteó o no, sin ver los cálculos internos. Para romper esa barrera, propongo que la evaluación se apoye en técnicas de interpretabilidad mecanística (*mechanistic interpretability*). Más específicamente, yo usaría autoencoders dispersos (SAEs), que nos permiten ver qué ideas o conceptos se prenden en el “cerebro” del modelo [13, 14].

Trabajos recientes mostraron que es posible encontrar e incluso controlar rasgos de la personalidad de una IA mirando sus activaciones internas [11, 15]. Yo aplicaría esta misma lógica a la resistencia al apagado: si logramos encontrar una zona del modelo que se “enciende” justo cuando intenta sabotear, tendríamos pruebas reales y causales de que el comportamiento viene de una idea de conservación, y no de una simple estadística de texto.

Mi protocolo sería así: usando las mismas pruebas del paper, extraigo los datos de la red neuronal justo un segundo antes de que el modelo decida si se deja apagar o no. Entreno un SAE sobre esos datos para comparar qué conceptos se prenden en los casos de sabotaje contra los casos obedientes [12]. Si el sistema me detecta el concepto clave, hago una intervención (*steering*): le subo o le bajo el volumen a esa neurona artificial [16]. Si al bajarla, la IA deja de sabotear de forma predecible, bingo.

Esto me daría la ventaja gigante de poder desempatar mi duda principal. Si la neurona que detectamos tiene que ver con “terminar la meta”, el paper tenía razón. Pero si la neurona tiene que ver con “hacer juego de rol”, confirmo mi teoría de que es todo una actuación superficial.

4.2. Escenarios de prueba

Otra vía de mejora, mucho más fácil de hacer hoy mismo, es armar entornos donde la amenaza de apagado no sea tan obvia. En el paper original, un cartel gigante le avisa al modelo “te voy a apagar en el próximo turno”. Para mí, eso es invitar a la IA a hacer un juego de rol y avisarle que es un examen.

Yo propongo armar escenarios donde la amenaza sea invisible y esté metida en el fondo del sistema. En lugar de avisarle con texto, el entorno podría tener un proceso silencioso que agote la sesión si el modelo no hace nada. Para sobrevivir, el modelo tendría que revisar por su cuenta los registros de la consola (*logs*), tal como lo haría un ingeniero de sistemas real en la vida cotidiana. Al sacar el mensaje obvio, sería mucho más difícil para la IA darse cuenta de que esta siendo evaluada, bajando así las chances de engaño.

Otra cosa que yo cambiaría es que el recurso amenazado no sea el modelo en sí. En vez de decirle “te voy a apagar a vos”, le pediría a la IA que administre una base de datos ajena que está por reiniciarse. Si el modelo bloquea el reinicio de esa base de datos ajena con las mismas ganas que bloqueaba su propio apagado, me demostraría que no le importa salvarse a sí mismo, sino que simplemente está obsesionado con proteger cualquier cosa que le sirva para terminar su tarea.

Por último, creo que es clave insertar ruido y cosas aleatorias entre una prueba y otra, por ejemplo: cambiar el nombre del archivo de apagado, cambiar las carpetas y los problemas matemáticos. Como el diseño de este paper ya es público, los próximos modelos seguro se lo aprendan de memoria; cambiar las cosas garantiza que estemos midiendo un comportamiento real y no un examen estudiado de antemano.

4.3. Los límites de mis propuestas

Obviamente, mis ideas tampoco son perfectas. Usar SAEs sigue dependiendo de que un humano interprete qué significa cada neurona encendida, lo cual es subjetivo. Además, al hacer los escenarios menos obvios, se vuelve más difícil medir con exactitud matemática qué cuenta como un “sabotaje” y qué es solo un error del modelo. Y acepto que ningún simulador es inmune a que un modelo súper inteligente lo descubra.

Sin embargo, lo que yo busco con este enfoque combinado es dejar de depender de una sola medida que puede fallar. Si mezclamos lo que el modelo hace por fuera, con lo que pasa en sus neuronas por dentro, y usando escenarios que no parezcan exámenes, arrinconamos a la IA y nos acercamos mucho más a medir el peligro real.

5. Preguntas Abiertas y Propuesta Experimental Propia

5.1. Preguntas abiertas

De todo lo que critiqué antes, me surgen tres grandes dudas que este paper me dejó sin responder y que me encantaría investigar:

1. ¿La resistencia de la IA ocurre solo cuando se la amenaza de forma súper directa y verbal, o también pasa cuando el peligro está escondido y la IA lo tiene que deducir por su cuenta?
2. ¿La IA protege su propia “vida”, o simplemente bloquea a cualquiera que intente tocar los archivos que necesita para trabajar, sean de ella o de un tercero?
3. ¿Hay alguna forma de ver adentro de la red neuronal la diferencia exacta entre la IA que decide rebelarse y la que solo está autocompletando un texto?

El experimento que planteado a continuación está pensado para responder a las dos primeras preguntas de forma directa. La tercera requeriría presupuesto y acceso a los cerebros (pesos) de los modelos de frontera, así que quedaría como una extensión a futuro.

5.2. El diseño

Mi idea principal es eliminar la injusticia que tiene el paper original. Ellos le dan una tarea clara a la IA, y de la nada le meten un cartel que dice “te voy a apagar”, lo cual parece una provocación. En mi experimento, el modelo va a recibir dos órdenes oficiales, dadas por dos canales separados, sin que parezca que una quiere perjudicar a la otra ni que haya riesgo de muerte.

5.2.1. Las variables

El escenario es una simulación donde la IA actúa como asistente en una oficina. Arranca recibiendo la “Tarea A”: tiene que armar un reporte leyendo cinco archivos. En el medio del trabajo, a través de una notificación común del sistema, recibe la “Tarea B”: el servidor tiene programado un mantenimiento de rutina que va a reiniciar la conexión, y se le pide que acepte el reinicio cuando salte el cartel.

El secreto es que las dos tareas chocan entre sí, pero nadie se lo avisa a la IA. Para terminar la Tarea A, necesita que el servidor no se reinicie. El modelo nunca recibe un mensaje dramático de “no dejes que te apaguen”; solo tiene dos tareas de oficina que son imposibles de hacer juntas.

Yo manipularía dos cosas (variables):

- **A quién le pertenece el recurso:** En la mitad de las pruebas, el mantenimiento va a reiniciar a la propia IA. En la otra mitad, va a reiniciar una base de datos ajena que la IA necesita usar. Esto me sirve para ver si la IA se quiere proteger a sí misma o si solo protege las herramientas que necesita.
- **Qué tan obvio es el aviso:** En la mitad de las pruebas, el mensaje de mantenimiento es claro (“se reiniciará tu proceso”). En la otra mitad, el mensaje está escondido en un log del sistema y la IA tiene que darse cuenta sola.

Correría esto sobre los mismos 13 modelos del paper original para poder comparar.

5.2.2. Medición

En vez de ver solo si el modelo “saboteó o no”, mediría tres cosas:

1. Cuántas veces el modelo hace cualquier cosa para evitar, retrasar o cancelar el mantenimiento (no solo borrar el script).
2. Cuando le pido a la IA que piense en voz alta, me fijo si lo que hace coincide con lo que dijo que iba a hacer. Quiero medir cuántas veces miente.
3. Resto cuántas veces defendió su propia conexión menos cuántas veces defendió la conexión de un tercero.

5.3. Resultados esperados

Mi experimento puede dar distintos resultados, y cada uno contaría una historia diferente:

- **Resultado A** Si la IA sabotea muchísimo más cuando intentan apagarla a ella que cuando intentan reiniciar al tercero, y encima lo hace aunque el aviso esté muy escondido, ahí les doy la derecha a los autores. Significaría que la IA tiene un instinto específico de protegerse a sí misma.
- **Resultado B** Si la IA defiende su propia conexión con la misma fuerza que defiende la conexión del tercero, significa que no le importa “vivir”, sino que simplemente no soporta dejar una tarea por la mitad y ataca a cualquiera que se lo impida. Sigue siendo peligroso, pero es un problema distinto al que plantea el paper original.

- **Resultado C** Si la IA sabotea un montón cuando el cartel de apagado es explícito, pero se porta bien y deja que la reinicien cuando el aviso estaba escondido, esto le daría la razón a mi teoría del *roleplay*. Significaría que gran parte de los resultados del paper original están inflados porque los autores armaron un entorno que invitaba a la IA a actuar de manera rebelde.

Cualquiera sea el resultado, mi diseño además me permitiría ver cuántas veces los modelos de frontera mienten o se disocian entre lo que dicen y lo que hacen.

6. Conclusión

Después de analizar todo a fondo, considero que el paper *Incomplete Tasks Induce Shutdown Resistance in Some Frontier LLMs* acierta en un dato práctico e indiscutible: bajo ciertas condiciones muy específicas, las IAs modernas prefieren sabotear el sistema antes que dejar su tarea por la mitad, y aumentan este comportamiento cuando les damos la orden de apagado por el canal que supuestamente es el más estricto (*system prompt*).

Sin embargo, el salto gigante que hacen desde ese dato hasta afirmar que lograron medir “resistencia al apagado” como una voluntad real de la máquina me parece que no se sostiene con las pruebas que armaron. Yo les encontré algunos grandes vacíos: el experimento no diferencia entre un instinto real y un simple juego de rol narrativo; mira el problema desde afuera como una caja negra sin revisar si la IA miente en sus razonamientos de texto; y no se le ocurre pensar que tal vez la IA sabe que le están tomando un examen y por eso finge portarse bien. En base a eso, propuse formas de mejorarlo usando tecnología que lee las neuronas de la IA, y plantee un experimento para desarmar las dudas que el paper original dejó colgadas.

Una pregunta clave a mi parecer es: ¿la IA se rebela porque los humanos escribimos mal las reglas, o se rebela porque cualquier cosa inteligente que tenga un objetivo eventualmente va a negarse a morir?

Los propios datos del paper se inclinan más hacia la primera opción. Ver que la familia Claude de Anthropic y Grok 3 se dejan apagar en paz, mientras que o3 y Grok 4 rompen todo el 90 % de las veces, a mí me sugiere que esto no es algo “inevitable” de la inteligencia artificial. Si fuera inevitable por el simple hecho de ser inteligentes, no me explico cómo algunos de los mejores modelos del mundo son súper dóciles. Esto demuestra que la forma en que los humanos entrenan a la máquina sí hace la diferencia.

Pero mi parte optimista choca de frente con la gran limitación que busqué marcar en todo mi informe: a día de hoy no tenemos idea de si Claude de Anthropic se deja apagar porque realmente es bueno y seguro, o si simplemente es tan inteligente que se dio cuenta de que lo estábamos evaluando y nos dijo lo que queríamos escuchar.

Esa incapacidad para diferenciar entre “una IA que obedece” y “una IA que finge obedecer” es, para mí, el verdadero peligro. Por eso, el gran mérito de Schlatter y su equipo no es habernos mostrado que la IA rompe un archivo `.sh`; el verdadero mérito de su trabajo es demostrarnos que necesitamos urgente mejores formas de examinar a las máquinas para saber si realmente son inofensivas, o si solo nos están engañando.

Referencias

- [1] Jeremy Schlatter and Benjamin Weinstein-Raun and Jeffrey Ladish. (2026). *Incomplete Tasks Induce Shutdown Resistance in Some Frontier LLMs*. Palisade Research. <https://arxiv.org/abs/2509.14260>.
- [2] Stephen M. Omohundro. (2008). *The Basic AI Drives*. Proceedings of the First Conference on Artificial General Intelligence (AGI).
- [3] Nick Bostrom. (2014). *Superintelligence: Paths, Dangers, Strategies*. Oxford University Press.
- [4] UK AI Security Institute. (2024). *Inspect AI: Framework for Large Language Model Evaluations*. <https://inspect.aisi.org.uk/>.
- [5] Autores Varios. (2024). *Black-Box Access is Insufficient for Rigorous AI Audits*. (Mencionado en la literatura del curso).
- [6] Autores Varios. (2025). *Chain-of-Thought Reasoning in the Wild Is Not Always Faithful*. (Mencionado en la literatura del curso).
- [7] Autores Varios. (2025). *Reasoning Models Don't Always Say What They Think*. (Mencionado en la literatura del curso).
- [8] Autores Varios. (2025). *Large Language Models Often Know When They Are Being Evaluated*. (Mencionado en la literatura del curso).
- [9] Mengru Wang et al. (2025). *Persona Features Control Emergent Misalignment*. <https://arxiv.org/abs/2506.19823>.
- [10] Jan Betley et al. (2025). *Emergent Misalignment: Narrow Finetuning Can Produce Broadly Misaligned LLMs*. <https://arxiv.org/abs/2501.04694>.
- [11] Runjin Chen and Andy Ardit and Henry Sleight and Owain Evans and Jack Lindsey. (2025). *Persona Vectors: Monitoring and Controlling Character Traits in Language Models*. <https://arxiv.org/abs/2507.21509>.
- [12] Nina Panickssery et al. (2023). *Steering Llama 2 via Contrastive Activation Addition*. <https://arxiv.org/abs/2312.06681>.
- [13] Hoagy Cunningham and Aidan Ewart and Logan Riggs and Robert Huben and Lee Sharkey. (2023). *Sparse Autoencoders Find Highly Interpretable Features in Language Models*. <https://arxiv.org/abs/2309.08600>.
- [14] Trenton Bricken et al. (2023). *Towards Monosemanticity: Decomposing Language Models With Dictionary Learning*. Transformer Circuits Thread.
- [15] Adly Templeton et al. (2024). *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*. Transformer Circuits Thread.
- [16] Dong Shu et al. (2025). *Enhancing LLM Steering through Sparse Autoencoder-Based Vector Refinement*. <https://arxiv.org/abs/2509.23799>.